Automatic Python Code Summarization Using Sequence-to-Sequence Models and Transformer Encoder–Decoder Networks

1. Introduction and Objectives

1.1 Background

Automatic source code summarization aims to generate short natural-language descriptions of code, improving readability, maintenance, onboarding, and documentation. In real-world repositories, code is often under-documented, making automatic summarization an important task in Machine Learning for Software Analysis (MLSA).

1.2 Problem Definition

The task is modeled as a **sequence-to-sequence (seq2seq) learning problem**:

- **Input**: Tokenized Python source code (function body)
- **Output**: Tokenized natural-language summary (docstring text used as label)

The system must learn to map code tokens to meaningful summary tokens.

1.3 Objectives

The primary goal of this project is to build a tool that generates natural language summaries for Python code. Code summarization helps developers understand code faster, facilitates better documentation, and improves overall software maintainability.

1.4 Main Research Challenge

A major risk in code summarization is **data leakage**, where docstrings/comments appear inside the input code. This can inflate evaluation scores because the model learns to copy the docstring instead of learning code semantics. Detecting and eliminating this leakage became a central methodological contribution of this project.

2. Technologies and Tools Used

2.1 Programming and Environment

- **Python 3** — Primary programming language
- **Google Colab** — GPU runtime for training
- **Google Drive** — Checkpoint persistence across sessions
- **GitHub** — Project organization and version control

2.2 Deep Learning Framework

- **PyTorch** — Core framework
  - `torch.nn` model components
  - `torch.nn.Transformer` encoder–decoder
  - `torch.optim.AdamW` optimizer
  - `torch.utils.data.DataLoader`

- `torch.nn.utils.clip_grad_norm_` for gradient clipping
- `torch.amp.autocast` and `torch.amp.GradScaler` for mixed precision

2.3 Data Processing

- **Hugging Face `datasets`** — `datasets.load_dataset`
- **Python AST** — `ast.parse` for validation, `ast.get_docstring` for leakage detection
- **Hugging Face `tokenizers`** — BPE tokenizer (`tokenizers.Tokenizer.from_file`)
- **JSONL format** — Processed dataset storage
- Standard libs: `json`, `os`, `sys`, `argparse`, `random`, `re`, `tqdm`

2.4 Evaluation Libraries

- **ROUGE**: `rouge_score.rouge_scorer`
- **BLEU**: `nltk.translate.bleu_score` + smoothing
- **METEOR**: `nltk.translate.meteor_score` + WordNet + Punkt
- **Perplexity**: `torch.nn.CrossEntropyLoss` + `math.exp()`

3. Methodology

3.1 Data

Dataset Evolution

This project used two datasets across different experimental stages:

Dataset A: CodeXGLUE Code-to-Text (Early Stage - LSTM Baseline)

For initial experiments with the LSTM baseline, we used the **google/code_x_glue_ct_code_to_text (Python)** dataset from Hugging Face. This dataset provides:

- `code`: Python function bodies
- `docstring`: Natural language descriptions

The data was preprocessed using `scripts/prepare_data.py` which:

- Extracted the first sentence of each docstring as the summary
- Applied length constraints: max 4000 chars for code, 10-400 chars for summary
- Removed exact duplicates by code text
- Saved cleaned data in JSONL format to `data/processed/`

However, this dataset still contained docstrings within the code, creating potential data leakage.

Example from CodeXGLUE (with docstring in code):

```
# Code:
def settext(self, text, cls='current'):
    """Set the text for this element.
```

```
    Arguments:
        text (str): The text
        cls (str): The class of the text, defaults to ``current``
    """
    self.replace(TextContent, value=text, cls=cls)
```

Extracted Summary: Set the text for this element.

Note: The docstring is visible inside the code, which causes data leakage during training.

Dataset B: AhmedSSoliman/CodeSearchNet (Final Stage - Transformer)

The final dataset used was **AhmedSSoliman/CodeSearchNet (Python)** from Hugging Face. This dataset provides:

- `code` : Python function bodies
- `docstring` : Natural language descriptions (used as target summaries)

This dataset was selected because:

- It is widely used in code summarization research
- It is compatible with modern Hugging Face tooling
- It supports building a leakage-free dataset through preprocessing

Example from CodeSearchNet (clean - no docstring in code):

```
# Code:
def on_log(self):
    def decorator(handler):
        self.client.on_log = handler
        return handler
    return decorator
```

Summary: Decorate a callback function to handle MQTT logging.

Note: The docstring is stored separately, not inside the code. This prevents data leakage.

Data Preprocessing

All datasets were transformed into a unified JSONL format:

```
{ "code": "...", "summary": "..." }
```

Stored in: `data/processed/codesearchnet_clean/` with `train.jsonl` , `validation.jsonl` , and `test.jsonl` splits.

Leakage-Free Filtering (Core Contribution)

Our preprocessing script applies strict filtering to prevent data leakage:

Code-side rules:

- Code must parse in Python 3 using `ast.parse(code)`
- Drop examples if input contains docstring: `ast.get_docstring(tree)`

Summary-side rules:

- Extract first line of cleaned summary from docstring
- Require ≥ 3 words
- Remove likely truncated summaries (e.g., starting with "After/Before/When")
- Remove incomplete punctuation endings (`: , ;`)

Dataset Sizes:

Commented Data (Dataset A - CodeXGLUE):

Split	Samples Kept
Train	248,029
Validation	13,663
Test	14,609

Partially Cleaned Data (Mixed Comments):

Split	Samples Kept
Train	185,271
Validation	10,384
Test	10,384

Tokenization

A **BPE (Byte Pair Encoding) tokenizer** was used to convert both code and summary text into subword tokens:

- Trained once and saved as `tokenizer.json`
- Reused in training, inference, and evaluation

Special tokens:

- `[BOS]` / `[EOS]` for sequence boundaries
- `[PAD]` for padding to fixed length

3.2 Model Architecture

Baseline Model: Seq2Seq LSTM + Attention

A baseline model serves as a reference for comparison to evaluate whether improved methods actually perform better.

Why LSTM was chosen initially:

- LSTM (Long Short-Term Memory) networks are a classic approach for sequence-to-sequence tasks
- Well-documented and widely used in early neural machine translation and summarization research
- Provides a solid foundation for understanding the task before moving to more complex architectures

Architecture:

- **Encoder**: BiLSTM (hidden size 256)
- **Decoder**: LSTM (hidden size 512)
- **Embedding size**: 256
- Luong multiplicative attention
- Teacher forcing during training

Why LSTM was slow: LSTM processes sequences **sequentially** — each token must wait for the previous token's computation to complete. This creates a fundamental bottleneck:

- Cannot parallelize across sequence positions during training
- Training time scales linearly with sequence length
- For our dataset, each epoch took approximately **365 seconds**
- GPU utilization is inefficient because most compute units sit idle waiting for sequential dependencies

Why we switched to Transformer:

- Transformers process all tokens **in parallel** using self-attention, dramatically reducing training time
- Better at capturing long-range dependencies in code (e.g., matching function calls to definitions)
- More efficient GPU utilization through parallelized matrix operations
- State-of-the-art results in code understanding tasks

Final Model: Transformer Encoder–Decoder

Our final model is implemented using `torch.nn.Transformer`, a custom model built in PyTorch (not a pretrained language model).
Key components:

- Token embeddings for source and target
- Sinusoidal positional encoding
- Causal decoder mask
- Padding masks

Configuration:

Parameter	Value
d_model	512
nhead	8
Encoder layers	6
Decoder layers	6
Feedforward dim	2048
Dropout	0.3

3.3 Training

Training Hyperparameters

Parameter	Value
Batch size	32
Max source length	256
Max target length	64
Optimizer	AdamW
Learning rate	3e-4
Weight decay	0.05
Gradient clipping	1.0
Max epochs	100
Early stopping patience	8

Parameter	Value
Early stopping min_delta	0.0005
LR scheduler	ReduceLROnPlateau

Subset sizes used per experiment: To handle dataset scale under Google Colab constraints, we employed stratified sampling by:

- Code length bucket
- Summary length bucket

Experiment	Train Samples	Validation Samples	Reason
Exp 1-3 (LSTM & initial Transformer)	50,000	10,000	Limited by LSTM's slow sequential processing
Exp 4 (Final Transformer)	100,000	10,000	Transformer's parallel processing allows more data

Checkpointing and Resume Support

Two checkpoints are saved:

- `best.pt` (best validation loss)
- `last.pt` (latest state for resume)

Saved to Google Drive: `/content/drive/MyDrive/mls-python-code-summarization/models`

3.4 Inference and Decoding

The inference pipeline:

- Loads tokenizer and trained checkpoint
- Ensures `[BOS]` / `[EOS]` are added to input code
- Pads input to fixed length
- Creates a source mask (`src_ids != pad_id`)
- Generates summary tokens autoregressively

Decoding Improvements:

- No-repeat n-gram blocking
- Repetition penalty
- Temperature sampling (adds diversity)

These techniques reduce repetition loops and improve fluency in generated summaries.

4. Results and Discussion

4.1 Experiment 1 — LSTM Baseline + Commented Dataset

Goal: Establish baseline and validate training pipeline
Dataset: Hugging Face commented dataset (leakage possible)

Metric	Value
Train Loss	0.0756
Val Loss	0.0967
Train Accuracy	98.42%
Val Accuracy	98.25%
ROUGE-1	0.8761
ROUGE-2	0.8124
ROUGE-L	0.8765

Discussion: Very high scores were obtained, but later analysis showed this was likely caused by docstring leakage (shortcut learning). These results are recorded but considered not representative of leakage-free summarization.

Example of "Shortcut Learning"

When docstrings were present in the input code during training, the model learned to simply extract the docstring instead of summarizing the logic. When the docstring was removed, the model's performance on the same code significantly degraded:

Case 1: Code with Docstring (Leakage)

```
>> Enter Code:
def add(a, b):
    """Return the sum of two numbers."""
    return a + b

Generated Summary:
Return the sum of two numbers .
```

Case 2: Same Code without Docstring

```
>> Enter Code:
def add(a, b):
    return a + b

Generated Summary:
return a + b b b ti b b
```

Case 3: Logic-based Inference (Partial success)

```
>> Enter Code:
def max_of_two(a, b):
    if a > b:
        return a
    return b

Generated Summary:
a > b
```

4.2 Experiment 2 — LSTM Baseline + Mixed Comments Dataset

Goal: Test LSTM on partially cleaned data
Dataset: CodeXGLUE with mixed comments (some docstrings removed, some remaining)

Metric	Value
Train Loss	4.8957
Val Loss	5.0832
Train Accuracy	28.78%
Val Accuracy	27.88%
Epoch Time	~365s
Best Epoch	23

Discussion: After removing docstrings from the code, LSTM performance dropped significantly. This confirms that earlier high scores were due to data leakage. The slow training time and modest accuracy motivated the switch to Transformer architecture.

Why only training metrics? For intermediate experiments, we tracked Train/Val Loss and Accuracy during training to:

- Monitor learning progress in real-time
- Detect overfitting early (when val loss increases while train loss decreases)
- Make quick decisions about model viability without expensive evaluation

Full evaluation (ROUGE, BLEU, METEOR) was not performed because the poor training metrics already indicated this approach was not promising enough to justify the additional compute cost.

4.3 Experiment 3 — Transformer + Mixed Comments Dataset

Goal: Evaluate Transformer on partially cleaned data
Dataset: Same mixed comments dataset

Metric	Value
Train Loss	4.4016
Val Loss	5.0384
Train Accuracy	29.16%
Val Accuracy	28.52%
Epochs	25

Discussion: Although Transformers are powerful, this experiment still used the partially cleaned data (with potential leakage). The training metrics showed the model was learning, but we decided not to run full evaluation because:

- The dataset still contained leakage issues that would invalidate ROUGE/BLEU scores
- Computing evaluation metrics on a flawed dataset would waste compute resources
- Our focus shifted to preparing a properly cleaned dataset for the final experiment

The training metrics (Loss and Accuracy) were sufficient to confirm the model was functional before moving to the fully clean dataset.

4.4 Experiment 4 — Transformer + Cleaned CodeSearchNet (Final)

Goal: Final leakage-free training aligned with real task
Dataset: Cleaned AhmedSSoliman/CodeSearchNet

Metric	Value
ROUGE-1	0.2574
ROUGE-2	0.0614
ROUGE-L	0.2295
BLEU	0.0384

Metric	Value
METEOR	0.1490
Perplexity	51.54

Discussion: Results are expected to be lower than leakage-based experiments because the model must infer semantics from code instead of copying. This makes the experiment scientifically valid and aligned with the expected task definition.

4.5 Evaluation Metrics Explained

Metric	Description
Cross-Entropy Loss	Measures the model's prediction error during training
ROUGE (1/2/L)	Measures n-gram overlap between prediction and reference. Limitation: penalizes paraphrases
BLEU	Measures precision of n-gram matches. Useful but stricter and often low for summarization
METEOR	Accounts for stemming and synonym matching; includes word-order penalty
Perplexity	exp(avg cross-entropy loss) — lower values indicate higher model confidence

4.6 Qualitative Evaluation

Below are examples of summaries generated by our model:

Input Code	Generated Summary	Expected Summary
`def find_max(lst): max_val = lst[0]; for item in lst: if item > max_val: max_val = item; return max_val`	"Find the maximum value of a list or equal to an item"	"Find the maximum value in a list"
`def calculate_average_score(student_grades): if len(student_grades) == 0: return 0.0; total_sum = 0; for grade in student_grades: total_sum = total_sum + grade;`	"Calculate the average total cost of a given amount"	"Calculate the average score from student grades"

Input Code	Generated Summary	Expected Summary
`average = total_sum / len(student_grades); return average`		

Analysis

The model captures **partial semantic meaning** but produces imperfect summaries:

- **find_max**: Correctly identifies "maximum value" and "list", but adds irrelevant phrase "or equal to an item"
- **calculate_average_score**: Correctly identifies "calculate" and "average", but confuses "grades" with "cost"

Why These Results Are Expected

This behavior is characteristic of **from-scratch transformer training** without pretrained weights:

1. **No prior language knowledge**: Unlike models like CodeT5 or CodeBERT, our model has no pretrained understanding of code semantics
2. **Limited training data**: 100,000 samples is relatively small for learning code-to-text translation
3. **Pattern matching vs. understanding**: The model learns surface-level patterns (variable names, function structure) rather than deep code comprehension
4. **Vocabulary confusion**: Similar tokens (cost/score, amount/average) get confused because embeddings are learned from scratch

The model demonstrates that a from-scratch transformer can learn **approximate code summarization** but struggles with precise semantic accuracy. This validates the importance of pretrained language models in code intelligence tasks.

4.7 Design Justifications

Choice	Justification
Transformer over LSTM	Better at capturing long-range dependencies in code
BPE Tokenization	Handles out-of-vocabulary tokens and code-specific tokens well
AdamW Optimizer	Better generalization through decoupled weight decay
Early Stopping	Prevents overfitting given limited compute resources
AST-based Filtering	Ensures syntactically valid Python and removes leakage

4.8 Limitations

1. **Compute Constraints**: Limited GPU time and memory on Google Colab forced subsampling rather than training on full dataset
2. **Dataset Noise**: Even after filtering, docstrings vary in style and quality; many summaries are generic
3. **Training From Scratch**: Without pretraining, the Transformer must learn code structure and language generation from scratch, typically yielding lower metrics

5. Conclusion

This project developed and evaluated neural models for Python code summarization. Early experiments produced high scores but were later found to be influenced by docstring leakage. After redefining the task correctly and implementing a robust AST-based cleaning pipeline, the final Transformer model was trained on a cleaned CodeSearchNet dataset under realistic assumptions.

Although automatic metrics decreased compared to leakage-based experiments, the final methodology is scientifically valid and aligns with academic standards. This highlights the importance of dataset correctness and proper evaluation in machine learning for software analysis tasks.

Key Contributions:

1. Identification and elimination of data leakage through AST-based preprocessing
2. Implementation of a custom Transformer encoder-decoder for code summarization
3. Comprehensive evaluation using multiple metrics (ROUGE, BLEU, METEOR, Perplexity)

6. References

1. Hugging Face Datasets Library: https://huggingface.co/docs/datasets
2. AhmedSSoliman/CodeSearchNet
   Dataset: https://huggingface.co/datasets/AhmedSSoliman/CodeSearchNet
3. PyTorch Documentation: https://pytorch.org/docs/stable/index.html
4. ROUGE Score Library: https://pypi.org/project/rouge-score/
5. NLTK BLEU and METEOR: https://www.nltk.org/

7. Appendix

7.1 Group Members

- **Sarah Ahmed Abushaala**
- **Mariia Al-Kafri**
- **Ahmad Sohail Najib**

GitHub Repository: https://github.com/sd45g/mls-python-code-summarization

7.2 Future Improvements

If more time/resources were available:

- Train on the full cleaned dataset (not subset)
- Fine-tune a pretrained model (e.g., CodeT5) for larger gains